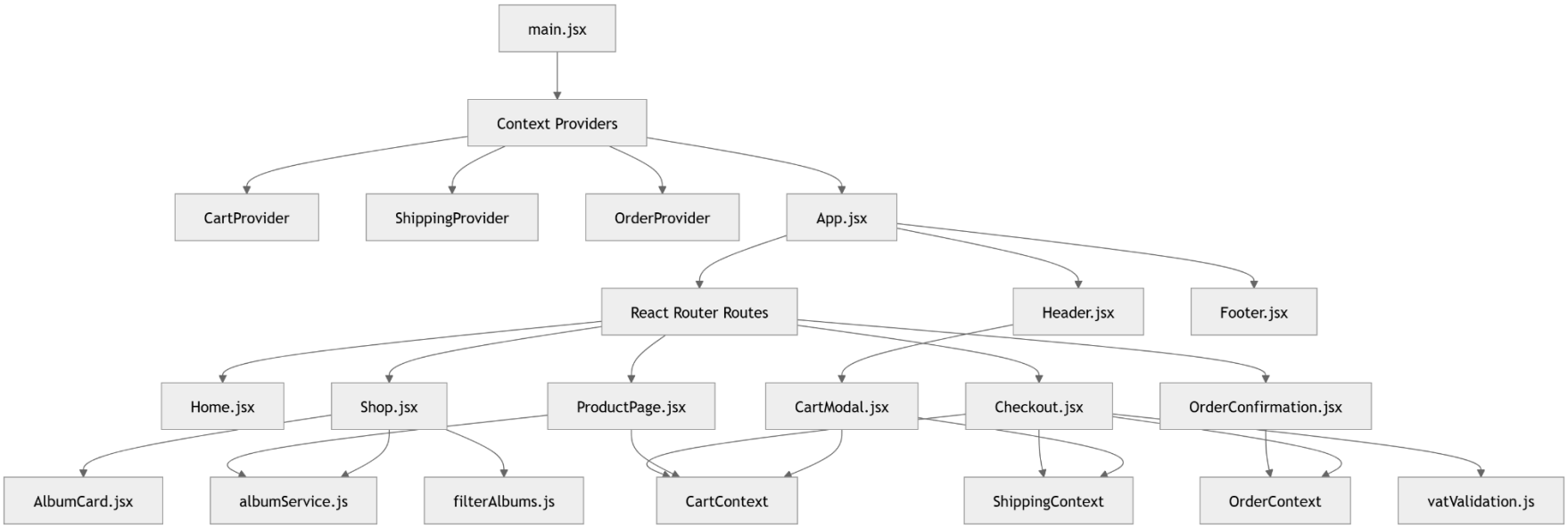
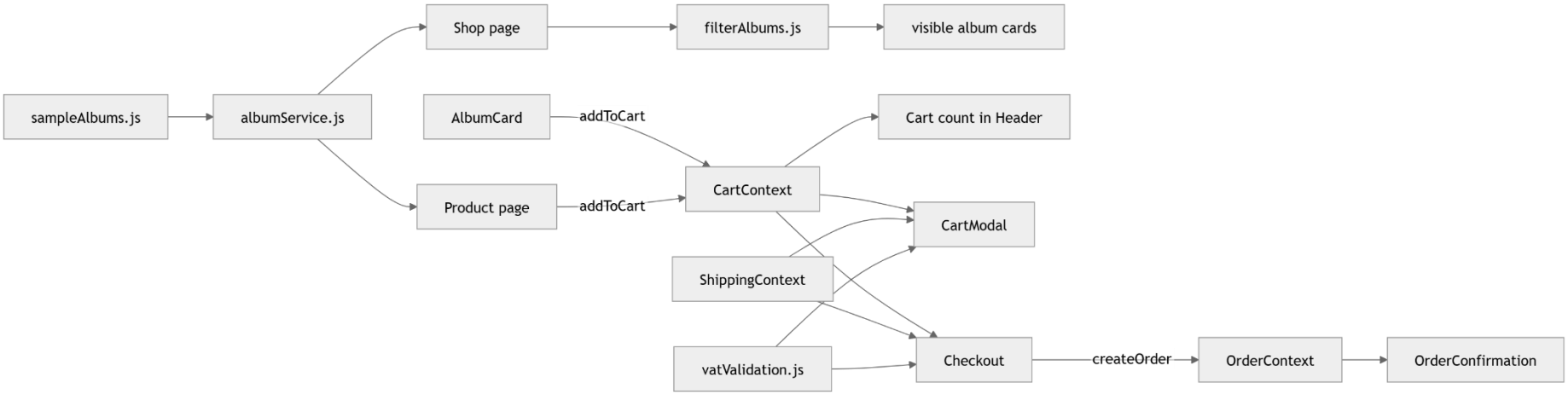
20260428 Vinyl Webshop, documentation notes (Jeff):



Big Picture Architecture



Runtime Data Flow

This is a React single-page webshop.

main.jsx starts the app and wraps it in three global state providers: cart, shipping, and order.

App.jsx defines the pages using React Router.

The shop page gets albums through albumService.js, filters them with filterAlbums.js, paginates them, and renders AlbumCard components.

The cart is global state, so both album cards and product detail pages can call addToCart(). The cart modal reads cart state and shipping state, calculates shipping/VAT totals, and sends the user to checkout.

The checkout page summarizes the order, applies discounts, and creates an order snapshot in OrderContext.

The confirmation page reads that last order and displays it after the cart has been cleared.

Key design idea - separation that will make the Firebase/backend swap much easier later.

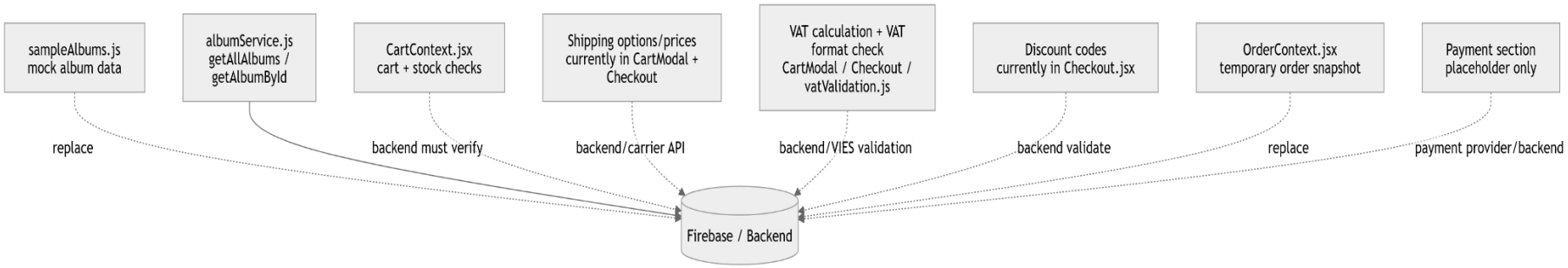
UI components should display and collect data.

Context files hold shared app state.

Service files hold reusable business logic.

Data files hold mock/backend-like data.

Backend handoff summary (how much of this will be implemented is to be determined)



The frontend currently **simulates** several backend responsibilities so the webshop can be tested end-to-end. These frontend pieces should later be replaced or verified by Firebase/backend logic.

What backend developers need to own

1. Album catalog
Frontend currently uses:

src/data/sampleAlbums.js
src/services/albumService.js

Backend should provide:

- album list
- album by ID
- image URLs
- price
- stock
- genre/category
- created/modified timestamps

2. Stock authority
Frontend currently prevents adding beyond stock, but backend must enforce final stock availability during checkout. Browser state cannot be trusted.

3. Shipping rates
Shipping prices are hard-coded in:

CartModal.jsx
Checkout.jsx

Backend should eventually return shipping options/prices based on:

- country
- address/postal code
- cart weight/quantity
- carrier choice

4. VAT
Frontend currently does format validation using:

src/services/vatValidation.js

Backend should validate EU VAT numbers through VIES and calculate final VAT legally.

5. Discounts
Frontend currently recognizes:

LEXICONELEV = 20%
ÖMER = 30%

Backend should validate discount codes, expiration, usage limits, and final discount amount.

6. Order creation
Frontend currently uses:

src/context/OrderContext.jsx

This is only a temporary in-memory confirmation. Backend should create a real order record in Firebase.

7. Payment
Frontend only has a placeholder. Backend/payment provider should handle:

- payment session
- payment confirmation
- secure callbacks/webhooks
- order status updates

Simple handoff sentence

“The React frontend is complete enough to simulate shopping, cart, shipping, VAT, discount, checkout, and confirmation. The backend needs to replace mock album data, verify stock/prices/VAT/discounts, create persistent orders, and handle payment securely.”